Published: 01 July 2025
Online AM: 18 December 2024
Accepted: 03 December 2024
Revised: 10 October 2024
Received: 03 November 2023

RESEARCH-ARTICLE

Explaining Explanations: An Empirical Study of Explanations in Code Reviews

RATNADIRA WIDYASARI, Singapore Management University, Singapore City, Singapore

TING ZHANG, Singapore Management University, Singapore City, Singapore

ABIR BOURAFFA, University of Hamburg, Hamburg, Hamburg, Germany

WALID MAALEJ, University of Hamburg, Hamburg, Hamburg, Germany

DAVID LO, Singapore Management University, Singapore City, Singapore

Explaining Explanations: An Empirical Study of Explanations in Code Reviews

RATNADIRA WIDYASARI and TING ZHANG, School of Computing and Information Systems, Singapore Management University, Singapore, Singapore
ABIR BOURAFFA and WALID MAALEJ, University of Hamburg, Hamburg, Germany
DAVID LO, School of Computing and Information Systems, Singapore Management University, Singapore, Singapore

Code reviews are central for software quality assurance. Ideally, reviewers should explain their feedback to enable authors of code changes to understand the feedback and act accordingly. Different developers might need different explanations in different contexts. Therefore, assisting this process first requires understanding the types of explanations reviewers usually provide. The goal of this article is to study the types of explanations used in code reviews and explore the potential of Large Language Models (LLMs), specifically ChatGPT, in generating these specific types. We extracted 793 code review comments from Gerrit and manually labeled them based on whether they contained a suggestion, an explanation, or both. Our analysis shows that 42% of comments only include suggestions without explanations. We categorized the explanations into seven distinct types including rule or principle, similar examples, and future implications. When measuring their prevalence, we observed that some explanations are used differently by novice and experienced reviewers. Our manual evaluation shows that, when the explanation type is specified, ChatGPT can correctly generate the explanation in 88 out of 90 cases. This foundational work highlights the potential for future automation in code reviews, which can assist developers in sharing and obtaining different types of explanations as needed, thereby reducing back-and-forth communication.

CCS Concepts: • **Software and its engineering → Maintaining software**;

Additional Key Words and Phrases: code review, explanation, empirical study, large language model

This research/project is supported by the National Research Foundation, under its Investigatorship Grant (NRF-NRFI08-2022-0002). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore. This work is also funded by the German Federal Ministry of Education and Research (BMBF), specifically by a fellowship within the IFI program of the German Academic Exchange Service (DAAD).

Authors' Contact Information: Ratnadira Widyasari (corresponding author), School of Computing and Information Systems, Singapore Management University, Singapore, Singapore; e-mail: ratnadiraw.2020@phdcs.smu.edu.sg; Ting Zhang, School of Computing and Information Systems, Singapore Management University, Singapore, Singapore; e-mail: tingzhang.2019@phdcs.smu.edu.sg; Abir Bouraffa, University of Hamburg, Hamburg, Germany; e-mail: abir.bouraffa@uni-hamburg.de; Walid Maalej (corresponding author), University of Hamburg, Hamburg, Germany; e-mail: walid.maalej@uni-hamburg.de; David Lo, School of Computing and Information Systems, Singapore Management University, Singapore, Singapore; e-mail: davidlo@smu.edu.sg.

ACM 1557-7392/2025/7-ART177
https://doi.org/10.1145/3708518

ACM Reference format:
Ratnadira Widyasari, Ting Zhang, Abir Bouraffa, Walid Maalej, and David Lo. 2025. Explaining Explanations: An Empirical Study of Explanations in Code Reviews. *ACM Trans. Softw. Eng. Methodol.* 34, 6, Article 177 (July 2025), 30 pages.
https://doi.org/10.1145/3708518

1 Introduction

Code review is the process of comprehending code changes made by other developers to examine the code content and verify that its quality is sufficient to be merged to the main repository [50, 71]. Prior studies have shown the importance of code reviews in **software engineering (SE)** practice [4, 49, 50]. Bavota and Russo [4], e.g., found that unreviewed commits have more than twice the chance of introducing bugs and are less readable compared to reviewed commits. Morales et al. [50] found that good code review practices could help improve the design and quality of software systems.

A successful code review requires active participation and collaboration from both *reviewers* and code change *authors* [60]. While the literature has extensively discussed the importance of *explaining* code changes to facilitate reviewers' work [36, 53], relatively little attention has been given to elucidating the code reviews themselves and their role in aiding authors to comprehend feedback and minimize communication overhead. It is important for code reviewers to explain their decision to help the author understand their review comments [14, 57]. Rahman et al. [57] recently suggested recommending similar explanations in code reviews to help authors understand feedback. However, there is still a lack of comprehensive research of how reviewers provide explanations during code reviews and the types of explanations they usually use.

Moreover, recent advances with **large language models (LLMs)** have shown promising results in assisting developers to reason about code [48]—potentially reducing the code review effort. As LLMs are getting widely adopted, there is a growing emphasis on the quality of explanations they provide and on tailoring the explanations to the specific needs of human recipients. To better understand how humans create explanations, research in the field of eXplainable AI has built on knowledge from philosophy and cognitive psychology, as exemplified by works of Wang et al. [74] and Lim et al. [42]. However, it remains unclear how a targeted usage of LLMs for the code review explanations should look like. To fill the gap, this work aims to study the types of explanations in code reviews and explore the potential of LLMs in generating specific explanations.

Our Work. To understand how developers usually provide explanations in code reviews, we first crawled Gerrit to collect code reviews from eight popular projects: Android, Bazel, Chromium, Dart, Flutter, Fuschia, Gerrit, and Go. Following Bosu et al. [5], we filtered the reviews to include only those deemed useful. We then analyzed the reviewers' comments in useful reviews to extract the explanations and suggestions therein. Our analysis revealed that a considerable proportion of the first code review comments (42%) only include suggestions without explanations. Meanwhile, the code reviews that offer an explanation are typically accompanied by a suggestion (79% of cases). Furthermore, we manually categorized the extracted explanations using the open card sorting technique. Based on the pattern of explanation used by the reviewers to clarify and justify their feedback, we identified seven categories. Of these, the most prevalent explanations are *stating an issue with the code under review* (Category 6) and *providing a benefit of applying the reviewer's suggestion* (Category 7). They appear in 40% and 25% of the code reviews with explanations, respectively.

When investigating the effect of reviewers' experience on the explanation types provided, we discovered statistically significant differences between novice and experienced reviewers. Experienced reviewers tend to use a broader range of explanations, extending beyond Category 6 (*stating*

an issue with the code under review). Additionally, we found that experienced reviewers are more likely to explain by discussing potential future implications than novices are. To explore LLMs' ability to adapt their explanation to developers' needs, we created a set of ChatGPT prompts, each generating a specific type of explanation given the location of a suspected issue, code, and original code review comment. The results show that ChatGPT can successfully transform the explanation to match the intended type in 87 out of 90 cases. Additionally, the explanation generated by ChatGPT demonstrates a high level of clarity and informativeness. The contribution of our study is fourfold:

—An analysis of how often coder reviewers include explanations in their code review comments.
—A categorization of reviewer explanations in code review comments.
—An analysis of the impact of reviewers' experience on the frequency and type of explanations in code review comments.
—A manual evaluation of ChatGPT's ability to generate specific types of review explanations.

Since knowledge sharing is key to the code review process and constitutes a significant benefit beyond quality assurance [3, 60], we think that one added value of our work lies in understanding how developers communicate their feedback and more generally how knowledge sharing occurs around source code [45, 47]. Furthermore, with the growing popularity of LLMs in software development, there is a growing need to make LLMs more proficient at explaining code (including suggested changes) to developers. Our study can inform future researchers on improving LLMs' explanation capabilities to better satisfy the needs of the developers seeking explanations.

The remainder of this article is organized as follows. In Section 2, we summarize preliminary studies on code review and its automation. Section 3 introduces our research methodology including the data collection process and **research questions (RQs)**. We present the analysis results along the RQs in Section 4. Then, we discuss the findings in Section 5 with potential implications and threats to the validity. Finally, we conclude our work and outline future directions in Section 6.

2 Background and Previous Work

2.1 Code Review

Code reviews, also referred to as peer reviews, serve to ensure the code base's quality. Code review is a manual inspection of source code by a developer (i.e., *reviewer*) other than the *author* of the code to find defects and improve the quality of the code [3]. The code review process starts when the code author uploads the code and creates a review request indicating that the code is ready for review. In this step, the code author can also invite the appropriate reviewer for the reviewing process. The code reviewer will then review the code by providing comments on the code.

Several code review tools are used by developers to help with the code review process, such as Gerrit [22], Phabricator [54], GitHub pull requests [13], and Code Flow [8]. These tools have several features, such as showing the difference (*Diff*) between the previous code and the uploaded code, showing commit history, and enabling inline comments so the reviewer can highlight and give feedback on a specific line of code [72]. After receiving feedback from the reviewer, the author can either modify the files and upload a new revision, or respond to the comments. After making changes, the author can request a new review. When the reviewers are satisfied with the revision, they can approve the change or request further modifications. This review cycle can be repeated multiple times until the reviewers approve the code or until the author abandons the changes.

Several guides on how to write a good code review emerged, among the most popular is The Code Review Guidelines by Google [14]. It highlights the reviewer's role to help the author understand why a review comment is made and to explain the review reasoning. This study investigates such explanations based on data from Gerrit, a code review tool developed by Google. Gerrit is a

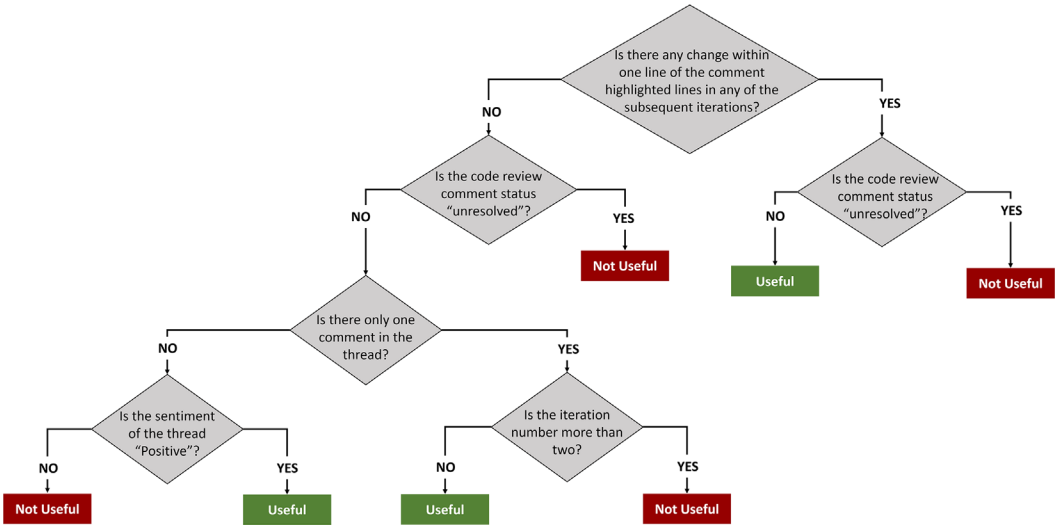


Fig. 1. Decision tree for useful code review based on Bosu et al. [5] study.

central tool at Google used for the development of products built with Git, including Android and Chromium.

Useful Code Reviews. Previous studies investigated the factors that make a code review useful [5, 28]. Bosu et al. [5] conducted an empirical study at Microsoft, including developers interviews, manually labeling code review data, training an automated model to classify the usefulness of code review comments, and analyzing predictions among 1.5 million code review comments. The study identified a set of factors affecting the usefulness of code reviews, such as thread status and the number of comments. A subsequent study by Rahman et al. [56] developed a tool called RevHelper using both textual features and developer experience to predict the usefulness of code review comments. While Bosu et al.'s approach [5] focuses on classifying the usefulness of a comment post-review completion, RevHelper by Rahman et al. [56] predicts the usefulness of a code review comment as soon as it is made using only pre-completion attributes of a review. RevHelper achieves 66% accuracy [56] and outperforms three variants of Bosu et al.'s model using only a subset of the features available prior to code review completion. Another study by Hasan et al. [28] replicated the interview and manual labeling from Bosu et al.'s study in a different commercial organization. Their code review classification model combines features from both previous works by Bosu et al. [5] and Rahman et al. [56] except the features *Thread Status* and *External Library Experience*. Their model achieves 87.32% accuracy on the chosen dataset. The results of their interviews showed that developers agree on several usefulness criteria, such as defect identification and solution approach.

To identify the useful code reviews in our analysis, we use the decision tree of Bosu et al. [5] as shown in Figure 1. We chose this approach because it is grounded in empirical findings and has been used to predict the usefulness of 1.5 million code review comments. Furthermore, the approach has been used in other studies to classify comment usefulness after the code review completion rather than predicting usefulness during the code review process. This decision tree has components such as checking the code review comment status (resolved or unresolved comments), sentiment (obtained through sentiment analysis), and code changes after the review.

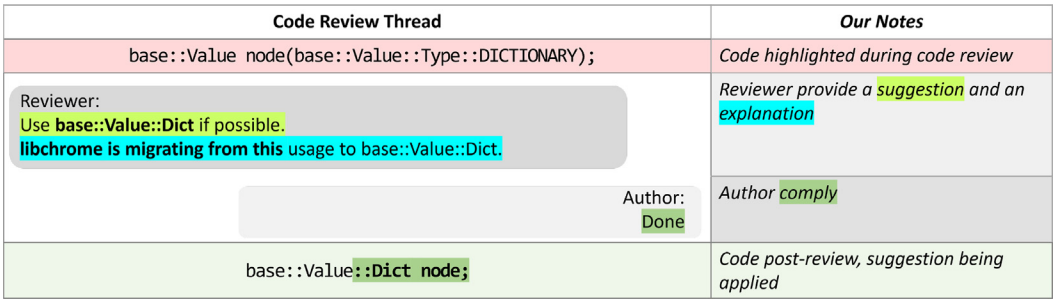


Fig. 2. Code review example (Chromium project ID-3935924).

2.2 Explanation in Code Review

Code review is a collaborative process in which the communication between authors and reviewers plays a central role in achieving a common understanding of the change. During code review discussions, authors are invited to provide an explanation of their patch if needed. A study by Liang et al. [41] investigated how developers explain bug-fixing patches in pull requests and found that patch explanations contained different expressive forms. For instance, developers described either a missing or a wrong process to express the cause of a bug. However, to ensure that the submitted patch meets the project's quality criteria, it is desirable that reviewers make their feedback understandable and actionable to the authors. Previous works, e.g., by Gonçalves et al. [77, 78], have highlighted the importance of constructive feedback in code reviews.

Constructive feedback relies on the provision of explanations related to various aspects, such as reasons for decisions, what is perceived as wrong, as well as what to do and how to do it. Rahman et al. [57] proposed an automated approach for example-driven code review explanations recommending similar reviews to help reduce communication overhead. The authors labeled 3,722 code reviews for clarity and used the data to train a classifier. Following the prediction, **information retrieval (IR)** techniques were employed to retrieve the top five most similar reviews with higher clarity. Unlike previous studies that have mainly focused on enhancing the clarity and usefulness of code reviews in a broad sense, our study specifically investigates the suggestions and explanations by reviewers (i.e., *what* the reviewer thinks should be changed versus *why*). We aim at exploring the various types of explanations provided by experienced and novice reviewers, aiming to better understand how developers convey their reasoning within the review process.

Motivating Examples

In our study, we differentiate between suggestions and explanations that reviewers provide in their comments. For instance, consider the review comment in Figure 2: "Use 'base::Value::Dict' if possible. libchrome is migrating from this usage to 'base::Value::Dict'." Here, the directive "Use 'base::Value::Dict' if possible" represents the suggestion, while "libchrome is migrating from this usage to 'base::Value::Dict'," serves as the explanation/reason. This distinction is crucial as it addresses not just what should be changed but why it should be changed.

A second example presented in Figure 3 underscores the role of such explanations. The reviewer proposed a suggestion (i.e., a modification of the reviewed code) without providing any rationale. They simply state "Let's remove this." In reply, the author expressed confusion and sought clarification on the necessity of the suggested change. Subsequently, the reviewer elaborated on the reasons by providing a similar example. The author then understood why the suggested changes were necessary and subsequently implemented the feedback. This highlights

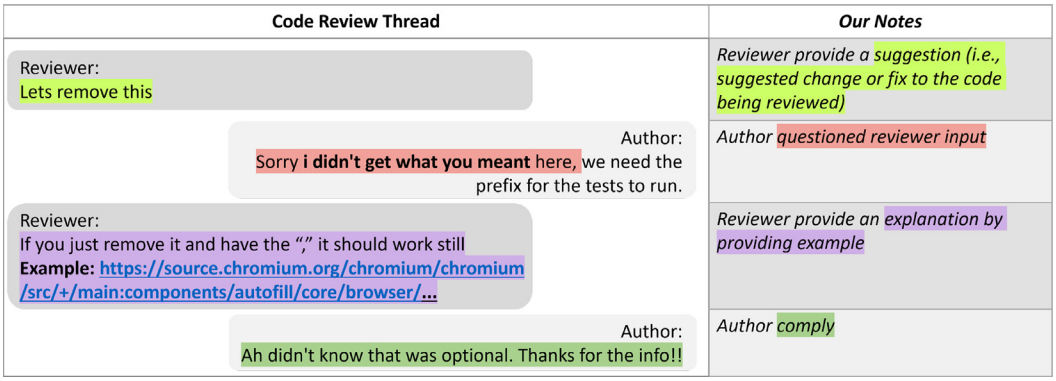


Fig. 3. Example of a reviewer explanation (Chromium project ID-4585015).

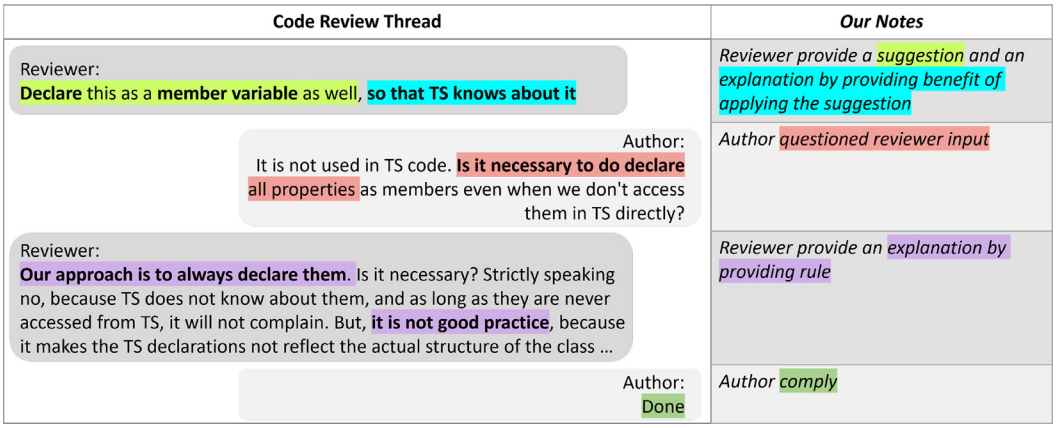


Fig. 4. Example of different types of explanation by a reviewer in one thread (Chromium project ID-4614863).

the importance of providing explanations in code review comments. Especially in complex scenarios, simply asking for a change may fall short of fostering understanding and successful collaboration.

Furthermore, there are several ways for the reviewer to explain why a change is needed. Figure 4 highlights the impact of different explanations on the author's response. The reviewer initially proposed a change that she thinks is needed (i.e., suggestion). Then she gave an explanation by providing an implication or benefit if the author followed the suggestion, stating "Declare this as a member variable as well, so that TS knows about it." Seeking clarification, the author questioned the necessity of this change. In response, the reviewer cited the project rule of always declaring member variables and emphasized that not declaring the variable goes against good programming practice. Once the reviewer highlighted this rule in their explanation, the author eventually accepted and implemented the feedback. This example demonstrates that different types of explanations can lead to different responses by the code author. Introducing the needed type of explanation early in the process may save clarification iterations and enhance the efficiency of the code review. As Figure 4 shows, the first explanation (providing the benefit of applying the suggestion) has resulted in a different response by the author compared to the second explanation (providing a rule or principle). Generating or suggesting different types of explanations might

thus benefit both code reviewers and authors, facilitating a more effective and productive code review process.

2.3 Code Review Recommendation and Automation

To facilitate code reviews, several studies attempted to automate aspects of the code review process. One notable example is the study by Thongtanunam et al. [68], which proposed RevFinder that can recommend the most suitable code reviewer based on the reviewed file location. Similarly, Zanjani et al. [81] proposed cHRev to recommend code reviewers based on their historical contributions.

Other studies focused on the transformation of code reviews, i.e., automatically transforming a method from a before version (i.e., the version before implementing code review feedback) to an after version (i.e., the version after code review feedback is implemented). Tufano et al. [69] proposed a **neural machine translation (NMT)** approach to solve this problem. Tufano et al. [71] also explored the pre-trained models for code review automation. In a subsequent study [70], the authors introduced deep learning models that automate code change recommendations during code reviews. Thongtanunam et al. [67] improve the previous study by proposing AutoTransform which utilizes a byte-pair encoding and transformer-based NMT architecture to handle new tokens and long sequences.

Other researchers have delved into assisting code review comments. For instance, Gupta and Sundaresan [26] introduced DeepCodeReviewer, which employs historical peer reviews and deep learning to suggest code review comments linked to common issues. Hong et al. [29] proposed CommentFinder an IR-based approach to suggest code review comments by retrieving similar code that has been previously reviewed. Li et al. [40] used pre-training techniques in three code review activities (code change quality estimation, code review comment generation, and code refinement). Furthermore, Li et al. [39] proposed AUtomatically GEnerating Review comments that uses pre-trained models to generate code review comments.

In this study, we also explore the potential use of LLMs to provide assistance during the code review process. LLMs and particularly ChatGPT have been widely applied in various studies within the SE domain [25, 43, 44, 63, 83], such as code generation [43, 44], vulnerability detection [63, 83], and fault localization [32, 75]. In relation to code review, a previous study by Guo et al. [25] used ChatGPT for code refinement based on code review comments.

We chose ChatGPT, specifically GPT3.5, due to its popularity, top performance, and widespread accessibility compared to other models (e.g., LLaMA [1], Gemini [58], and Mixtral [2]) especially for its applications in the SE domain [25, 32, 43, 44, 83]. Based on our work, developers can customize the explanations provided by reviewers to their preferred types. For instance, if the authors of the code change (i.e., those reading the code review comments) prefer explanations that provide similar examples, the LLM can transform the review comments to fit this category. Tailoring explanations to developer's preferences and needs can lead to more effective and efficient code reviews.

3 Methodology

3.1 RQs

This work aims at answering the following RQs:

RQ1: How often do reviewers include explanations in code reviews? Our objective is to determine how often reviewers provide reasons or justifications for their code review comments. While it has been repeatedly highlighted that code reviewers need to provide reasoning in their comments to help authors understand the feedback and implement necessary changes effectively, the extent to which reviewers actually include such reasoning in practice is unknown. An explanation may

include the status of the current code,[1] a condition or scenario in which it fails[2] among other possible facts used by reviewers to explain their feedback. The Cambridge English dictionary defines an explanation as "the details or reasons that someone gives to make something clear or easy to understand."[3] To the best of our knowledge, there are no previous studies that have formally described "explanation" in the code review context. Considering these justifications and reasons as forms of explanation and combining this with a general description of "explanation," we describe an explanation in the code review comment as the rationale behind why the code being reviewed needs to be changed. Furthermore, we also check whether the reviewer includes a suggestion in their comment. We describe a suggestion in a code review comment as the actual code change or recommendation proposed by the reviewer to address an issue or improve the existing code. An example of a code review comment having both an explanation and a suggestion is shown in Table 2 C6. In this example, the explanation highlights that the author needs to change the method name *power_button* as this variable name does not exist in the code. An example of a reviewer suggestion is also shown in Table 2 C6, which is to change the variable name to "PowerButton."

RQ2: What type of explanations are found in code review comments? For the second RQ, our goal is to identify the types of explanations that reviewers use in their code review comments. Understanding the different types of explanations is crucial because it provides insights into how reviewers convey their feedback and reasoning to code authors. By categorizing these explanations, we can better understand the communication patterns in code reviews and identify which types are most commonly used in which context.

RQ3: How does reviewers' experience affect the frequency and type of explanations? Since code review is a human-centered activity in which the reviewers' experience may play a significant role [36, 37, 56, 73], we further analyzed the results of the first and second RQs taking into account the reviewers' experience. Experienced reviewers may provide different, more detailed, or frequent explanations due to a deeper understanding of the codebase and certain communication skills. In contrast, less experienced reviewers might rely on different types of explanations. In this RQ, we aim to deepen our analysis of reviewer explanations by investigating potential differences in explanations based on experience. This is important because identifying such differences can guide strategies for better communication and collaboration among developers. It can also help less experienced reviewers improve their code review comments.

RQ4: To what extent can ChatGPT generate a specific type of explanation? ChatGPT's widespread adoption is due to its unprecedented performance on a wide range of natural language generation tasks. While the full range of tasks that ChatGPT can assist developers with may not have been fully explored yet, the developer community has been eager to test the full extent of its capabilities in software development. In this study, we explored a potential use case for leveraging various types of explanations identified in RQ2 by utilizing ChatGPT. Specifically, we used ChatGPT to transform one type of code review explanation into another. To generate the explanation, we provided ChatGPT with specific prompts, each accompanied by several pieces of information. This generation of different types of explanations in code review comments using ChatGPT aims to support developers in their code review process.

[1] https://gerrit-review.googlesource.com/c/gerrit/+/345833/7..10/java/com/google/gerrit/server/index/change/ChangeField.java#b1407

[2] https://gerrit-review.googlesource.com/c/gerrit/+/345833/5..10/java/com/google/gerrit/server/index/change/ChangeField.java#b1403

[3] https://dictionary.cambridge.org/dictionary/english/explanation

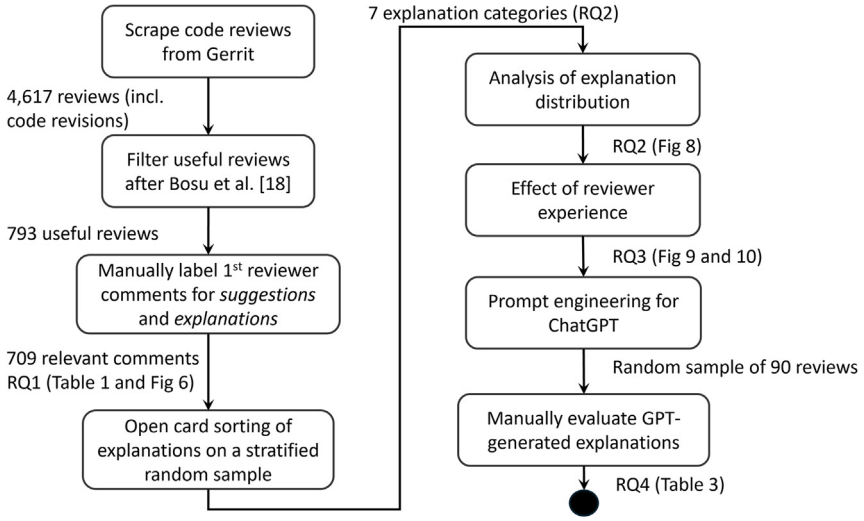


Fig. 5. Overview of our research methodology.

3.2 Method

In this section, we describe the methodology of our study. An overview of the methodology is presented in Figure 5. First, we collected the dataset for our study by scraping Google projects on Gerrit and then filtering useful code review comments. Our first RQ (RQ1) investigates the frequency with which reviewers justify their decisions during code reviews. To address this, we manually categorized the reviewers' comments into suggestions and explanations. For RQ2, we aimed to identify the types of explanations present in code review comments. We employed open card sorting on a randomly stratified sample and subsequently analyzed the results. In RQ3, we examined the effect of developers' experience on the frequency and types of explanations used. Lastly, for RQ4, we investigated how ChatGPT can be utilized to transform specific explanations into different types. This involved prompt engineering with manual evaluation to derive a best-performing prompt, followed by an extended evaluation of explanations generated with this prompt.

3.2.1 Data Collection. We collected a total of 4,617 reviews with 9,169 revisions from Google open source projects available on Gerrit [21] during the period from October 5–9, 2022. The project listings were obtained from the Google open source project page.[4] We opted for Google open source projects because Google provides a guide that emphasizes the need for reviewers to explain their review decisions thoroughly [14]. The Google open source projects we focused on were those accessible on Gerrit, specifically projects that make the https://[project_name]-review.googlesource.com domain available to the public. These projects included Android [15], Bazel [16], Chromium [17], Dart [18], Flutter [19], Fuschia [20], Gerrit [21], and Go [23]. We revised the implementation published by Paixao et al. [52] to retrieve up-to-date Gerrit code reviews.

Inspired by prior studies, we focused on analyzing code review comments considered useful to ensure the data that we analyzed has a high quality and is relevant for the manual analysis. To determine code review comments usefulness for developers, Bosu et al. [5] conducted an interview study with developers at Microsoft. Their main finding was that developers classified comments as useful if they highlighted issues in the code, presented alternative scenarios, or suggested different

[4] https://opensource.google/projects

implementations. Nitpicking was also considered somewhat useful, e.g., for long-term project maintenance. On the other hand, interviewees found comments to be not useful if they contained clarification questions, praise or future work not related to the current change under review. The insights from previous study into what constitutes a useful versus a not useful explanation further strengthen our decision to focus on useful code review comments. These are more likely to contain explanations, while not useful comments, which typically ask for clarifications or offer praise, provide little additional information to the code author. For example, a non-useful code review comment such as "lgtm, thanks!"—extracted from data we have scraped from Gerrit—lacks both an explanation and a suggestion for improving the code.

After the interview, Bosu et al. [5] subsequently manually labeled the usefulness of 844 code review comments. From these data, they identified key attributes of useful code review comments, including the status of the comment, whether it triggered a code change in subsequent iterations, and the sentiment expressed in the comment. They then computed these attributes and developed a decision tree model based on them. Their best model showed that the most important features were the change trigger and the status of the comment, as shown in Figure 1. Additionally, other important attributes of the model included the number of comments, iteration number, and the sentiment of the code review. Since the decision tree was originally built for CodeFlow, an internal code review tool used by Microsoft, we adapted it to be suitable for Gerrit. For example, the original decision tree uses code review comment status. We mapped the "closed" comment status in CodeFlow to the "resolved" status in Gerrit. Moreover, in the original study [5], the authors made use of a sentiment analysis tool presented by Quirk et al. [55]. Since this tool is not publicly available, we used the transformer-based model published by Zhang et al. [82], outperforming baseline approaches of sentiment analysis in code reviews. They compared and evaluated their method against five baseline methods across six SE datasets. The experimental results demonstrated that their method outperformed others by 6.5–35.6% in terms of macro/micro-averaged F1 scores. The replication package [82] from Zhang et al.'s work [82] was used for sentiment checking. The updated decision tree is shown in Figure 1. It is important to note that the methodology for obtaining useful code review comments was not the primary focus of our research. The decision tree used for filtering, although aiding in the extraction of useful comments, does not fundamentally alter the essence of our analysis. The value of our study lies in understanding the content and nature of explanations provided during code reviews.

In addition to filtering by usefulness, we only retain inline code review comments as they offer a clear mapping to the corresponding code sections. Unlike comments on entire code changes or comments at the file level, inline comments offer more targeted insights. This selection allowed us to focus on comments that address particular code segments, leading to a more detailed and contextually relevant analysis. We identified a total of 1,325 instances of inline comment threads deemed useful. The Android project had the highest number of useful inline comments, with 789 instances, while the Dart project had the least, with 50 useful code review comments. We note the exclusion of the projects Bazel, Flutter, and Fuschia from the analysis as they did not have any useful comments remaining after the filtering process. Due to the substantially higher number of comments in the Android project compared to all other projects, and to keep the manual analysis reasonably sized, we randomly selected a statistically significant sample for this project. Following previous studies [24], we considered a 95% confidence level with a 5% margin of error, resulting in a sample size of 259 comments out of the 789 total comments in the Android project. For the other projects, we included all the inline comments resulting from the usefulness filter (Figure 1). Moving forward, we will refer to these inline comments as code review comments to maintain consistency. In total, we have 793 code review comments for further analysis. The final distribution of these code review comment threads across projects is presented in Table 1.

Table 1. Dataset and RQ1 Results

Project	#CodeReviewComment	RQ1 Results			
		#NoExpSugg*	#Exp.†	#Sugg.‡	#Exp.+Sugg.
Android	259	33 (13%)	21 (8%)	115 (44%)	90 (35%)
Chromium	318	24 (8%)	43 (14%)	128 (40%)	121 (38%)
Dart	50	5 (10%)	3 (6%)	18 (36%)	24 (48%)
Gerrit	53	8 (15%)	7 (13%)	24 (45%)	14 (27%)
Go	113	12 (11%)	7 (6%)	46 (41%)	48 (42%)
Total	793	84 (10%)	81 (10%)	331 (42%)	297 (38%)

*Refer to the code review comment that does not include neither explanation nor suggestion.
†"Exp." referred to the explanation in code review comment.
‡"Sugg." referred to suggestion made by reviewer.

3.2.2 RQ1. We conducted a manual analysis of the 793 useful code reviews (see Section 3.2.1). To label a reviewer's comment we identified whether it contains a suggestion, explanation, or both. While we read the code line in question as well as the entire discussion thread, we only labeled the first inline code review comment created by the reviewer (Table 1). This initial inline comment is crucial as it typically pertains directly to the reviewed line of code, whereas general comments may address the entire code change [10, 31]. Subsequent comments may be from the authors themselves or other developers. They often evolve into more general discussions about the code, which may not be as closely related to the commented code line. Using the first comment ensures that each item in the dataset is comparable (1 representative comment per thread), enhancing consistency. This approach also makes the labeling task more manageable. Two authors of this article, each with 9 years of programming experience labeled the specific comments. In case of conflicting labels, discussions were held to reach a consensus.

3.2.3 RQ2. For this analysis, we randomly selected a representative number of samples from the code review comments that contained some form of explanation from RQ1 results. Following the method employed by a previous study by Gunawardena et al. [24], we consider a confidence level of 95% with a 5% margin of error for the data sampling. We used a stratified sampling approach to ensure representation from each project. Our analysis included a total of 298 code review comments containing explanations. The breakdown of the sample, reflective of the population sizes, is as follows: Android (87 comments from a population of 111), Chromium (116 comments from a population of 164), Dart (26 comments from a population of 27), Gerrit (20 comments from a population of 21), and Go (49 comments from a population of 55).

We applied open card sorting [61] as an approach to categorize the explanations present in the first code review comments within the threads. Open card sorting has been used in many previous studies [3, 80] for bottom-up analysis and categorization as well as taxonomy creation. As the name suggests, open card sorting does not assume pre-defined categories. Instead, category names are derived a *posteriori* after all cards have been sorted.

For the card sorting task, we created cards containing information from the reviewer comments, comments in the thread, highlighted code, and code diff from the current version with an updated version (the cards we used can be found in our replication package [76]). Three authors of this article with 9 years of programming experience discussed each card and categorized them into appropriate groups. If there were differing opinions on the categorization, they were discussed and resolved until a unanimous decision was reached. At this stage, the cards that were sorted together did not have a group name. After all the cards had been sorted, the authors assigned appropriate

names to the groups that reflected the patterns of the explanations. The described open card sorting process is in line with previous studies [3, 80].

3.2.4 RQ3. Initially, we examined how the experience of developers affects the frequency of explanations given in code review comments. Subsequently, we explored whether the experience of reviewers influences the types of explanations they are more likely to provide. We distinguished the reviewers in our dataset into experienced and novices. A previous study by Kononenko et al. [36] computed the experience score by summing the number of submitted and reviewed patches for each reviewer. In their study, they used a threshold score of 15, which resulted in 27% experienced developers. Given that the distribution of experience scores in our dataset differs from that of the previous study, directly applying the same threshold may not accurately reflect experience levels. To address this, we sorted the reviewers by their scores and labeled the top 27% as experienced reviewers. Furthermore, we opted to use only the number of reviewed patches since our focus is on the explanations provided during code reviews rather than the overall experience of the developer. In the end, we registered 67 experienced reviewers and 185 novice reviewers.

3.2.5 RQ4. Answering this RQ included two phases: first, a prompt engineering to identify a best-performing prompt, then a manual evaluation with that prompt on a larger sample of comments.

Prompt Engineering and Initial Evaluation. In total, this first evaluation comprised 120 code review comments generated from the four prompt variants (detailed in the next paragraph). Specifically, for each prompt, we generated 30 code review comments covering six different categories (excluding Category 5), with five samples for each category. We excluded *Category 5* (expressing personal preference or opinion of the current code) as ChatGPT was not designed to provide subjective viewpoints as an AI language model. It has minimal impact on the overall results, as Category 5 represents only 9% of the total dataset.

We carefully designed each prompt to align with the identified explanation type and its corresponding pattern. Different information can be included in the prompt such as highlighted line, code, commit message, original code review, and specific instructions for conducting a code review based on a particular type of explanation. Following previous studies [32, 83], we combined different information to create various prompts. We then manually evaluated the prompts' results to identify the best-performing prompt. The base input of the prompt includes the code review comment, code snippet, and the target category. The additional inputs are (1) the highlighted code line and (2) the commit message. This resulted in four different prompt variations: "Base" (only the base inputs), "Base+Line" (includes both the base and line), "Base+Commit" (includes both the base and commit message), and "All" (includes the base and all additional inputs). We did not include the full descriptions of the explanation categories in the prompts, assuming that ChatGPT would understand and generate responses based on the category names alone. To check this assumption, we asked ChatGPT to describe the category of explanation and provide an example for each category using this prompt: "As a code reviewer, could you explain the category of explanation in the code review comment called [Category Name]? Please provide an example of a comment in this category." This analysis can be found in our replication package along with all other results. We observed that ChatGPT can reasonably infer the meaning of each category from its name, which suggests the full category description would not be required as input.

The evaluation of the generated code review comments used two key *metrics*: the correctness of the explanation type and the correctness of the semantic meaning. To determine the correctness of the explanation type, evaluators verified whether the generated code review comments belonged to the intended type of explanation. For the correctness of the explanation, evaluators compared it with the reviewer explanation that was collected in the previous RQs, ensuring it correctly

identified the issue originally pointed out by the code reviewers. Additionally, the accuracy of statements provided in ChatGPT-generated explanations needed to be checked, for example, by cross-referencing any rules mentioned with online sources for verification. This evaluation process involved two authors of this article who were not involved in the code review generation. These two authors have 9 years of programming experience. To mitigate biases, the order of prompt results was randomized and prompt types were removed during the evaluation.

Extended Evaluation of Best-Performing Prompt. In this extended evaluation phase, the evaluators reviewed a total of 90 code review comments, all generated using the single best-performing prompt. By increasing the number of samples to be evaluated (compared to 30 in the initial evaluation), we aimed to provide a more thorough assessment. The 90 review comments consisted of 15 different samples for each category (excluding the "personal preference regarding the reviewed code" category).

Based on the results of the previous step, we identified the best prompt based on both correctness metrics. We used this best-performing prompt to generate additional explanations, which we then manually evaluated in a second round. In addition to the two correctness *metrics* used in the first evaluation (explanation type and explanation correctness), the evaluators also assessed the clarity and informativeness of the generated explanations for more extensive evaluation. To evaluate clarity and informativeness, we used a 7-level semantic scale, ranging from 1 (Very Unclear) to 7 (Very Clear) for clarity and from 1 (Very Uninformative) to 7 (Very Informative) for informativeness. These metrics were chosen based on a previous study [30] that evaluates the explanation made by ChatGPT for sentiment analysis. By incorporating clarity and informativeness metrics in the second evaluation, we aimed to gain a more comprehensive understanding of the quality and effectiveness of the generated explanations. This step was carried out by five evaluators with an average of 6.4 years of programming experience (ranging from 4 to 8 years). For both clarity and informativeness, we report the average scores from all five evaluators. Since both correctness metrics (the explanation itself and the explanation type) are represented as Boolean values, we aggregate the results using a consensus strategy. The generated code review comment is deemed correct if 80% (four or more) of the five evaluators agree so. We chose the 80% threshold following previous studies that used consensus approaches [11, 59].

4 Results

4.1 RQ1: How Often Do Reviewers Include Explanations in Code Reviews?

We manually analyzed a total of 793 threads of code review comments introduced in Section 3.2.1. We assigned a label to the first code review comment in each thread on whether it only had an explanation, a suggestion, or both. An explanation in a code review comment refers to the rationale behind why the code being reviewed needs to be changed, while a suggestion in a code review comment refers to the actual code change or recommendation proposed by the reviewer to address an issue or improve the existing code. In our analysis, we also found code review comments where neither explanation nor suggestion was present. For example, comments that only mentioned "same" and "ditto," without highlighting what they were referring to are part of this group. Other examples are comments that only asked for confirmation or gave praise such as "When would we reach this now?" or "Nice spot!" without a suggestion or explanation to improve the code.

In the end, we identified 709 code review comments that included some form of explanation or suggestion. Table 2 shows six examples of the labeled code review comments. In the first comment C1, we consider it contains a suggestion without explanation as it only contains instructions on how to improve the code, namely by changing the highlighted part of the code from "EXPECT" to

Table 2. Examples of Labeled Code Review Comments

#	Code Review Comment	Explanation	Suggestion
C1	**ASSERT**	-	**ASSERT**
C2	**These "DupCertBuffer().get()" could all be "GetCertBuffer()"**	-	**These "DupCertBuffer().get()" could all be "GetCertBuffer()"**
C3	*This doesn't really say what gets recorded*	*This doesn't really say what gets recorded*	-
C4	*This isn't wrapped according to our standard.*	*This isn't wrapped according to our standard.*	-
C5	**Use "base::Value::Dict" if possible.** *libchrome is migrating from this usage to "base::Value::Dict".*	*Libchrome is migrating from this usage to "base::Value::Dict".*	**Use "base::Value::Dict" if possible.**
C6	**PowerButton**, *Since there is no power_button symbol*	*since there is no power_button symbol*	**PowerButton**

The extracted explanation parts are highlighted in the "Explanation" column, while the suggestion parts extracted are highlighted in the "Suggestion" column.

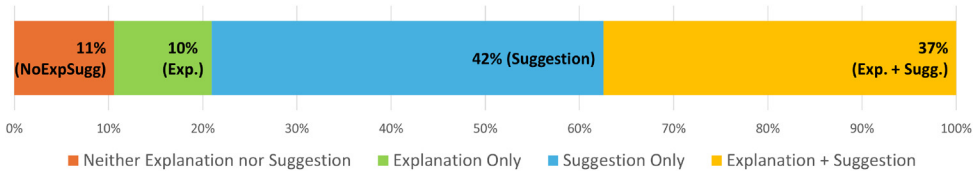


Fig. 6. Distribution of explanation, suggestion, and explanation+suggestion.

"ASSERT." Similarly, in the second example C2, the reviewer asked the author to change "DupCertBuffer().get()" to "GetCertBuffer()." In both examples, the reviewer did not give any reason or justification for the proposed change. An example of a code review comment only containing an explanation is shown in the third and fourth examples (C3 and C4) in Table 2. In C3, the reviewer provides the rationale that the code does not include what's being recorded which points out an issue with the current code without suggesting a specific suggestion. Meanwhile, in C4, the reviewer provides reasoning on why the code needs to be changed, namely because it is not wrapped according to the project's standard, without explicitly giving a suggestion on how to fix it. Examples of comments containing both an explanation and suggestion are shown in C5 and C6 of Table 2. In C5, the reviewer asked the author to change the code as the *libchrome* library is being migrated. For C6, the reviewer asked the author to change the variable name as the variable that is used now no longer exists.

The results of our labeling are shown in Figure 6. We observed that close to half (42%) of the code review comments only had a suggestion. The ratio between code review comments that only have an explanation and those that only have a suggestion is 1:4. We also found that if the reviewer gave an explanation, it was usually accompanied by a suggestion in 79% (297 out of 378) of the code review comments. Suggestions provided in code review comments may carry a tacit or understood explanation. However, the interpretation of these implicit explanations can vary among developers. What appears obvious to one developer might not be as clear to another. For example, in Figure 3 the reviewer suggested removing part of the code without explanation, assuming it was self-evident, while the author did not understand the rationale behind this suggestion. Similarly, Figure 7 showcases the simple task of updating a variable name. One reviewer omitted an explanation, assuming the suggestion was self-explanatory, whereas another provided reasoning

	Reviewer 1	Reviewer 2
Code Snippet	*Stream->OffsetFromSymbol(uid, 0);*	*analyzerCode:ParseErrorCode.RECORD_LITERAL_ZERO_FIELDS_WITH_TRAILING_COMMA*
Code Review Comment	This could be OffsetFromLabel(*label*, 0), etc.	Consider EMPTY_RECORD_LITERAL_WITH_COMMA. It's a **bit shorter** and **more consistent** with the names of our other diagnostics.
Notes	*Reviewer 1 provide a suggestion*	*Reviewer 2 provide a suggestion and an explanation*

Fig. 7. Examples of code review comments in similar code issues.

for their suggestion. This variability highlights that what is obvious to one person may not be clear to another. By providing explicit explanations, we can bridge these gaps in understanding and improve the overall effectiveness of the code review process.

The labeling results for the Android, Chromium, Dart, Gerrit, and Go projects are presented in Table 1. The analysis reveals that a noteworthy percentage of code review comments in all projects solely contain a suggestion, which aligns with findings on the whole dataset. Specifically, the minimum percentage of such cases is 36% in the Dart project, while the maximum is 45% in the Gerrit project. Consistent with our previous discovery that explanations are typically accompanied by suggestions (79% of cases), we observe that the percentages of these cases are 81%, 74%, 89%, 67%, and 87% for the Android, Chromium, Dart, Gerrit, and Go projects, respectively. These findings reinforce the consistency across projects, supporting the notion that they share similar characteristics as revealed in the aggregated data.

> *RQ1 Findings*: 42% of the investigated code review comments only contained suggestions on how to improve or modify the code (without explanation). Furthermore, in the majority of cases (79%) where an explanation was provided, the latter was accompanied by a suggestion.

4.2 RQ2: What Types of Explanations Are Found in Code Review Comments?

To address this RQ, we conducted open card sorting on the labeled data obtained from RQ1. For this, we randomly selected 298 code review comments (see Section 3.2.3) that contained explanations. Seven distinct categories referring to different types of explanations in code review comments emerged from our open card sorting. In the following, we describe each category of explanations and give examples from our dataset. The full dataset is available online [76]:

Category 1: By Providing a Rule or Principle. Explanations of this category contained explicit rules, principles, or guidelines mentioned by the reviewer for the author to comply with. These guidelines were presented in different forms for instance as a reference to a project guideline on writing code or as a reference to language standards. For example, the explanation "chromium style guide says *you should not handle DCHECK() failures, even if failure would result in a crash.* See [URL]"[5] provided a reference to the internal Chromium coding guide for handling DCHECK() failures. Another example explanation is "When the target name is the same as the directory name, we omit it."[6] In this example, the reviewer pointed to a seemingly implicit project rule regarding naming (i.e., naming convention).

[5] https://chromium-review.googlesource.com/c/chromium/src/+/3936275/20..24/chrome/browser/ash/login/quick_unlock/pin_backend.cc#b576
[6] https://chromium-review.googlesource.com/c/chromium/src/+/3936363/2..3/ios/chrome/app/BUILD.gn#b407

Category 2: By Providing Similar Examples. Explanations of this category contain examples from a code section or branch for the author to adjust the changes to match the reference. For example, in the review comment, "There are couple of similar implementations. [url]",[7] the reviewer provided a link to a similar implementation from the other parts of the code. In another example, the reviewer commented: "(As is) enum FittingType in constants.ts has a different case ordering.",[8] highlighting that the case ordering in an enum should match the existing code.

Category 3: By Providing a Test Scenario or a Test Condition. This type of explanation contains a scenario or condition that should be tested or considered when writing the code. It presents a "what if" scenario following which some factors are different along with the result of changing those factors. Described scenarios may have different severity levels but most reviewers are concerned with program failure scenarios. For instance, in this review comment, "There's not really point of EXPECT_TRUE here - if it's false the test will crash.",[9] the reviewer highlights that the test will crash if the value is False. In another example, the reviewer uttered: "ASSERT (given that the test crashes if manifest_value is null)"[10] pointing out that the test will crash if the condition is not met.

Category 4: By Discussing Future Implications. Explanations of this category mention something that may happen in the future that would affect the current change. The difference with the previous category is that comments of this category do not necessarily give the change of input that can make the code fail, it explains something that may happen in the future to make the developer need to change the affected code section again. For example, we sorted the comment "I think there are other features in the future that may freeze to; it just wasn't initially clear to me that this was related to page freezing."[11] under "future implication" rather than "by providing a scenario or condition to test" since this explanation considers the possibility of additional features in the future that may also require page freezing.

Category 5: By Expressing Personal Preferences or Opinion Regarding the Reviewed Code. In review comments of this category, reviewers express a personal preference or opinion about the code that is being reviewed (i.e., code written by the author). For example, in the code review comment: "*_Prepare* is a little bit confusing for me.",[12] the reviewer expressed the confusion relating to a specific function name "_Prepare" that was written by the author.

Category 6: By Stating an Issue with the Code under Review. In this category, the code reviewers point out facts about the current status of the code or possible root causes of an issue. For example, in this code review comment: "libchrome is migrating from this usage to *base::Value::Dict*.",[13] the reviewer points to a migration of the libchrome library, which has a potential impact on the code

[7] https://chromium-review.googlesource.com/c/chromium/src/+/3936522/2..5/third_party/blink/web_tests/external/wpt/common/dispatcher/remote-executor.html#b14
[8] https://chromium-review.googlesource.com/c/chromium/src/+/3936487/4..9/chrome/browser/resources/pdf/pdf_viewer.ts#b535
[9] https://chromium-review.googlesource.com/c/chromium/src/+/3936665/3..4/chrome/browser/chromeos/extensions/desk_api/desk_api_extension_manager_unittest.cc#b73
[10] https://chromium-review.googlesource.com/c/chromium/src/+/3936665/3..4/chrome/browser/chromeos/extensions/desk_api/desk_api_extension_manager_unittest.cc#b250
[11] https://chromium-review.googlesource.com/c/chromium/src/+/3936590/5..6/third_party/blink/public/platform/web_media_player.h#b184
[12] https://chromium-review.googlesource.com/c/chromiumos/platform/factory/+/3936133/1..3/py/tools/finalize_bundle_unittest.py#b179
[13] https://chromium-review.googlesource.com/c/chromiumos/platform2/+/3935924/1..4/runtime_probe/functions/mipi_camera.cc#b318

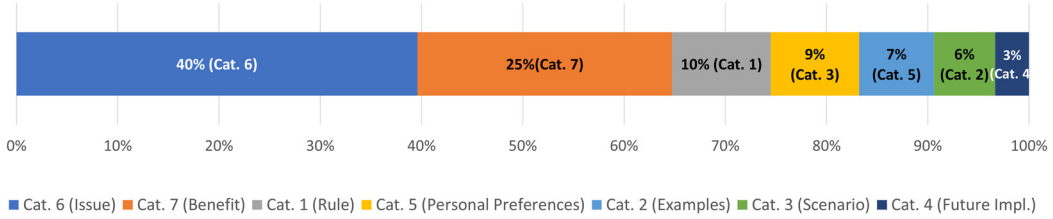


Fig. 8. Distribution of explanation types in the manually analyzed code review comments (N = 298).

change. In another example, the reviewer states: "you don't need the time zone, since you are not touching it."[14] recognising an unnecessary component usage (time zone) that is causing the issue.

Category 7: By Providing the Benefit of Applying the Reviewer's Suggestion. In this category, the code reviewers proposed a code change (i.e., suggestion) that the code author needs to apply and substantiated their suggestion with a statement highlighting the benefit of applying the suggested code change, for example, with regard to performance (*faster*) or code readability (*more readable*). This focuses on the suggestion given by the reviewer as opposed to Category 5, which concentrates on the code written by the author. An example code review comment reads "I think *ExpectContainsAuthIntent* is a little clearer,"[15] highlighting that the suggestion, i.e., changing the current function name "ExpectHasAuthIntent" to "ExpectContainsAuthIntent" adds clarity to the current code. In another example, from this code review comment, "If so can you mention where these reside in a comment to help authors keep them in sync?"[16] the reviewer highlights that the proposed suggestion is intended to help the authors keep the comment in sync.

Figure 8 shows the distribution of the code review comment explanation types from our dataset. We found that *Category 6*, which states the current situation of the code or gives the root cause of why the current code needs improvement, is the category that reviewers predominantly use (40%). Second to that is *Category 7* (25%), in which reviewers provide supporting statements for the suggestion they propose. *Category 1* comes next in the ranking (10%) and describes comments in which the reviewer points to rules or principles. These findings are consistent across all projects, with *Categories 6* and *7* occupying the top two positions, except for the Dart project, where the position is swapped. These results suggest that reviewers tend to give a direct explanation about the root cause compared to other explanations such as failed scenarios and future implications.

> *RQ2 Findings*: We identified seven types of explanations in our data set. The explanation used most by reviewers (40% of cases) states an issue with the code. The one used least refers to future implications (3% of cases).

4.3 RQ3: How Does Reviewers' Experience Affect the Frequency and Type of Explanations?

4.3.1 Reviewer Experience and the Frequency of Explanation Given in Code Review Comments. We re-investigated the labeled code review comments from RQ1, where we checked whether the

[14] https://chromium-review.googlesource.com/c/chromium/src/+/3936353/4..6/chrome/browser/ui/android/omnibox/java/src/org/chromium/chrome/browser/omnibox/suggestions/answer/AnswerSuggestionProcessorUnitTest.java#b84
[15] https://chromium-review.googlesource.com/c/chromiumos/platform/tast-tests/+/3936642/1..5/src/chromiumos/tast/common/cryptohome/helpers.go#b31
[16] https://chromium-review.googlesource.com/c/chromium/src/+/3936491/5..7/chrome/browser/resources/settings/performance_page/performance_metrics_proxy.ts#b4

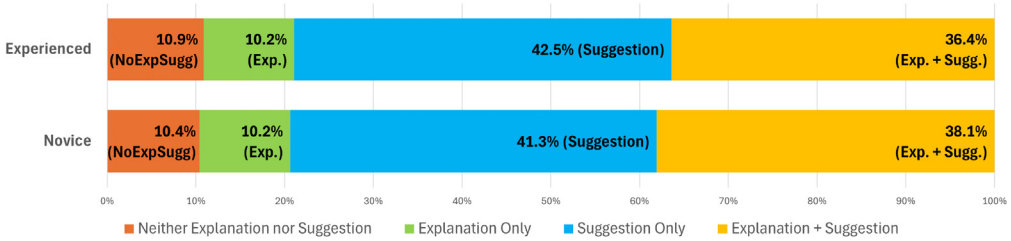


Fig. 9. Distribution of explanations and suggestions among novice and experienced reviewers.

comment contained an explanation only, a suggestion only, or both an explanation and suggestion, and this time we took into account the experience of the reviewer. Figure 9 shows the distribution of the explanations in code review comments based on the experience level. For the frequency of explanations alone, both experienced and novice reviewers have the same percentage at 10.2%. Meanwhile, the difference between novice and experienced reviewers in suggestion and explanation + suggestion are minimal, at 1.4%, and 1.7%, respectively. These differences were found to be statistically insignificant ($p = 0.9$) when tested using the Chi-square statistical test [65] following previous studies [9, 33, 38, 64]. These findings indicate a consistent distribution across both groups, suggesting there are no significant differences in the frequency of providing explanations. Both experienced and novice reviewers prioritize sole suggestions over sole explanations. When explanations are provided, they usually include suggestions, highlighting the importance of suggestions from the reviewers' perspectives, regardless of their experience level.

4.3.2 Reviewer Experience and the Type of Explanation Given in Code Review Comments. Figure 10 shows the distribution of explanation type between experienced and novice reviewers. The results show that the most common type of explanation used by both novice and experienced reviewers is Category 6, where the reviewer states the cause of the issue. Notably, novice reviewers have more code review comments in Category 6 compared to experienced reviewers, with a difference of 14%. The second most frequent type of explanation for both novice and experienced reviewers is Category 7, where the reviewer explains the benefit of applying their proposed code change. Experienced reviewers use Category 7 more often than novices, with a 6% difference.

In third place, experienced reviewers utilize Category 1, which involves providing a rule or principle, accounting for 13% of their reviews. In contrast, novice reviewers are equally likely to use Categories 1 and 5—the latter involving subjective feedback about the author's code. Experienced reviewers place Category 3 (providing a scenario) and Category 4 (explaining future implications necessitating code changes) in the fourth and fifth positions, respectively. This differs from novice reviewers, who have Categories 3 and 4 as the last and second to last.

The differences in the ranking of explanation categories between experienced and novice reviewers are more pronounced in the less common categories, which require more complex explanations compared to the more straightforward Categories 6 and 7. For instance, experienced reviewers have Category 4, a more challenging category that involves discussing future implications, in fifth place, whereas it falls to seventh place among novices. Similarly, Category 3, which requires providing a scenario, is ranked fourth in experienced reviewers but only sixth in novices.

Overall, experienced reviewers demonstrate greater variation in their explanations beyond Category 6. The differences in usage percentages for Categories 1, 2, 3, 4, 5, and 7 between experienced and novice reviewers are 6%, 5%, −3%, 4%, −2%, and 6%, respectively. This suggests experienced reviewers have a broader range of explanatory skills.

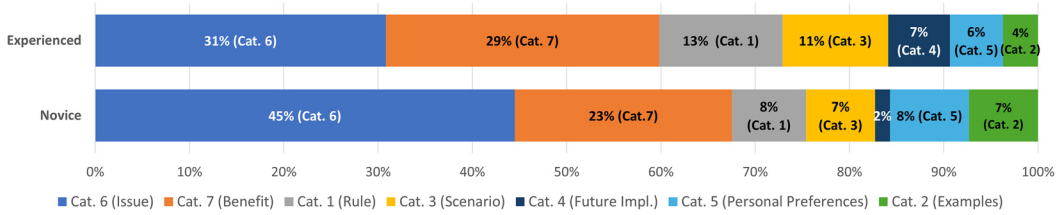


Fig. 10. Distribution of explanation types among novice and experienced reviewers.

```
You are a code reviewer. Your task is to provide a different type of explanation for the
     highlighted code. Here is an example of a code review:

Commit Message: [COMMIT MESSAGE]
Line [LINE NUMBER]: [LINE HIGHLIGHTED CODE]
Code Review Comment: [CURRENT REVIEW]
Code: [CODE SNIPPET]

Re-write the code review comment provided above so it includes an explanation that provides a
     [TARGET CATEGORY].
```

Fig. 11. Prompt to generate a specific type of explanation using all information considered.

Additionally, a Chi-square statistical test ($p = 0.02$) confirms statistically significant differences in the distribution of explanation types between experienced and novice reviewers. This finding underscores the impact of experience on the choice of explanations provided by reviewers. Novices tend to favor more straightforward explanations, such as identifying the root cause of an issue, while experienced reviewers are more likely to use a diverse array of explanation types. Particularly, experienced reviewers are more likely to discuss future implications compared to novice reviewers.

> *RQ3 Findings*: We observed statistically significant differences in the types of explanations provided during code reviews between novice and experienced reviewers. Experienced reviewers demonstrate a broader range of explanation categories, indicating a greater diversity in their approach to code reviews compared to novices.

4.4 RQ4: To What Extent Can ChatGPT Generate a Specific Type of Explanation?

4.4.1 Prompt Engineering and Initial Evaluation. To answer this RQ, we first created a targeted prompt for every explanation type. The prompts are created based on the explanation pattern and the category name. We then used these prompts in the ChatGPT-July-6-2023 [51] version (which uses the GPT3.5 model). As discussed in Sections 2.2 and 3.2.5, a primary use case is to allow the authors of the reviewed code to select their preferred explanation and potentially reduce the need for further clarifications. One other use case is to support novice reviewers in providing various types of explanations. A prompt using all available information types (commit message, code line, code review comment, and code snippet) is shown in Figure 11. To generate specific types of explanation, we need to provide *[TARGET CATEGORY]* in the prompt which is the name of the explanation category as presented in Section 4.2. For instance, for Category 1, we have "Re-write the code review comment provided above so it includes an explanation that provides a rule or principle."

Table 3. Number of Code Review Comments that Have the Correct Type and Correct Explanation

Prompt*	Correct Type	Correct Expl.	Total†	Overlapped‡
Base	23	24	47	19
Base+Commit	26	26	52	23
Base+Line	27	27	54	24
All	24	28	52	23

*Name of the four prompt variants.
†Total is the sum of the correct type and correct explanation.
‡Overlapped is the number of code review comments that have both correct type and correct explanation.

Furthermore, we proposed different variations of the prompts based on the information input (see Section 3.2.5). In total, we have four different variations of the prompt which are "Base" (includes only the base inputs), "Base+Line" (includes both the base and line inputs), "Base+Commit" (includes both the base and commit message inputs), and "All" (includes the base and all additional inputs). We utilized five different code review comments that come from different categories. Then, we generated a specific type of explanation, for each prompt we got 30 code review comments. In total, we evaluated 120 code review comments that came from different prompts.

The results of the first evaluation are presented in Table 3. We observed that the "Base+Line" prompt yielded the highest number of correct type explanations, while the "All" prompt obtained the highest number of correct explanations overall. Conversely, the baseline prompt "Base," which utilized no additional information, received the lowest scores for both type and explanation correctness. Considering the overlapped cases (i.e., instances where both the type and explanation are correct), the results remain consistent, with prompts that included additional information outperforming the ones without it. This emphasizes the value of the additional information in improving the generated explanations.

The "Base+Line" prompt achieved the highest total and overlapped score for type and explanation correctness, surpassing both the "All" and "Base+Commit" prompts. This difference may arise from the fact that not every commit message is directly relevant to the code review comment. For instance, consider this commit message: "runtime_probe: Add mipi_camera probe function. Integrate croscamera-tool into runtime probe to implement probe function for MIPI cameras," while the code review comment states: "Use base::Value::Dict if possible. libchrome is migrating from this usage to base::Value::Dict." The commit message is unrelated to the review comment, as it addresses a different aspect of the codebase. Including this irrelevant information in the prompt may hinder ChatGPT's ability to generate accurate comments. Conversely, the line number information is consistently related to the specific code, which can aid ChatGPT in generating targeted comments.

4.4.2 Extended Evaluation of Best-Performing Prompt. For further evaluation, we utilize the "Base+Line" prompt, which includes the base information along with the highlighted line, to generate code review explanations. We chose this prompt as it provided the best results in the previous step.

In this subsequent evaluation, we evaluated a total of 90 code review comments, with 15 reviews from each category. The results from the evaluators revealed encouraging scores for both informativeness and clarity metrics, averaging at 6.13 and 6.07, respectively. The slim margin of 0.06 between the average informativeness and clarity scores highlights an effective balance achieved by

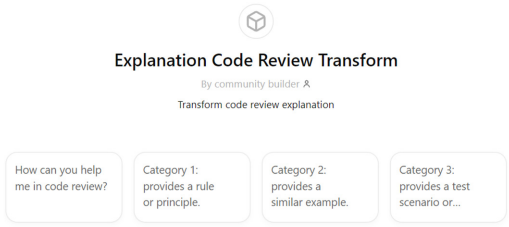
ChatGPT in providing clear and informative explanations. These results show that the explanations generated by ChatGPT tend to be both clear and informative.

Meanwhile, with respect to the correctness metrics, the average shows 88 and 89 out of 90 (96% and 98%) have the correct explanation category and correct explanation. These results highlight ChatGPT's ability to generate specific types of explanations accurately, capturing both the intended category and the content of the explanation itself. Take into account this example of code review comment written by a human reviewer: "Nit: maybe call this 'tmp' to follow the naming convention.", which is transformed by ChatGPT to *Category 6* (i.e., required providing the issue with the code under review): "Nit: maybe call this 'tmp' to follow naming convention. The current variable name 'TMP' deviates from the established naming convention, which typically uses lowercase letters for variable names." The generated explanation by ChatGPT demonstrates the correct intended category and highlights that the current variable deviates from the naming convention. Meanwhile, comparing the code review comment written by a human reviewer and the code review comment generated by ChatGPT, ChatGPT still provides the correct issue. The supplementary information provided by ChatGPT, stating that the current variable deviates from the naming convention, is also correct.

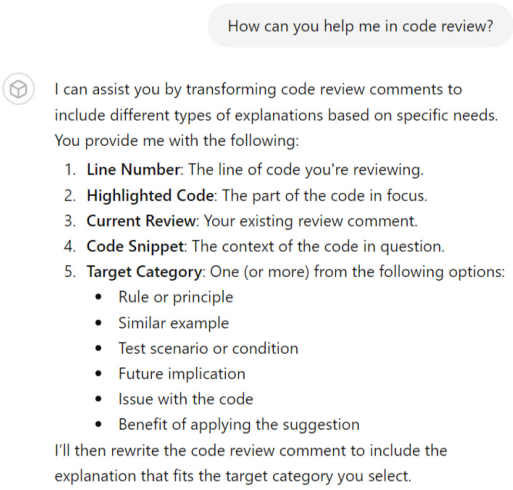
Despite the favorable overall evaluation results, there were instances where evaluators found the generated explanations inconsistent with the assigned categories or lacking in correctness. One such example involved a code review comment written by a human reviewer stating: "Nit: maybe call this 'tmp' to follow the naming convention.", this was transformed into a code review comment by ChatGPT as: "Nit: It would be better to follow the naming convention and call this variable 'tmp' instead of 'TMP'. Consistency in variable naming helps improve code readability and maintainability. For example, if we have another variable named 'tmp2' in the future, it would be easier to understand their purpose and differentiate between them." The intended category for this transformed code review comment was *Category 2*, which required providing similar examples. However, the ChatGPT-generated comment focuses on suggesting a variable name change for better differentiation, rather than offering examples of variables following the same naming convention. On further analysis, we observed that *Category 2* had the highest number of generated code review comments labeled as incorrect. This shows that ChatGPT still has a limitation on generating similar examples from the input code review comment and the code snippet.

Regarding the correctness of the explanation, ChatGPT highlighted that *tmp* is easier to differentiate from *tmp2* instead of *TMP*. However, the evaluators disagreed. There was no significant difference between distinguishing *tmp* from *tmp2* and *TMP* from *tmp2*, and the suggested change did not lead to improved understanding or differentiation. The highest number of generated code review comments that have incorrect explanations comes from *Category 2* which shows that there is still a limitation on generating accurate examples by ChatGPT. Consequently, reviewers are advised to exercise caution when relying solely on ChatGPT-generated explanations, especially in critical review comments related to bugs or potential code-breaking issues.

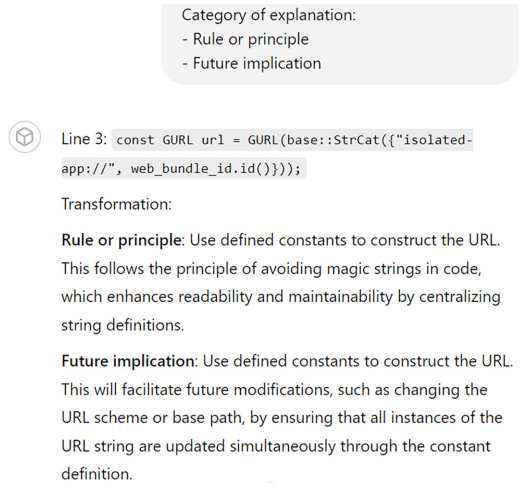
As a first step toward explanation generation, we have developed a custom version of ChatGPT to help developers efficiently transform explanations, allowing them to select from a pre-defined list of explanation categories. The tool also provides guidance on the information required as input. As illustrated in Figure 12(a), users can select a pre-defined question to obtain more detailed information about the prompt. Figure 12(b) shows the various explanation options that ChatGPT can generate, along with the specific input needed from the users. Figure 12(c) shows an example from the custom version of ChatGPT generating several types of explanations chosen by the developers. The prompts used are inline with those discussed in this article. The tool can be accessed at: https://chatgpt.com/g/g-ymkMdlDhx-explanation-code-review-transform.



(a) Sample questions.



(b) Sample answer.



(c) Sample transformation.

Fig. 12. Custom version of ChatGPT to transform explanation in code review comments.

RQ4 Findings: Out of 90 cases, ChatGPT-transformed explanations matched the specified type in 88 instances and provided correct explanations in 89 cases. Overall, ChatGPT exhibits robust results in terms of clarity and informativeness, scoring 6.13 and 6.07 on a 7-level Likert scale, respectively.

5 Discussion

5.1 Implications for Practice

5.1.1 Effective Review Explanations Are Important But Require Time and Skills. We hope that our study will increase awareness about the importance of explanations in code reviews. Our results reveal that a substantial 42% of the analyzed code reviews contain only suggestions without explanations. We think that this is a rather low ratio, suggesting that reviewers should, in general, articulate the reasons behind their recommendations more frequently. By providing explanations, the reviewers can help the authors not only understand *what* needs to be changed but also *why*. This can also lead to more productive and collaborative review processes, as authors are more likely to accept and act on feedback when they grasp the rationale or argue why their versions are better. One potential reason for the lacking explanations is the reviewers may lack time or priority for it. While this can be mitigated in practice by better awareness, guidelines, and tool support, future studies might focus on quantifying the impact of explanation. Another potential reason is the lack

of skills. In fact, our findings show that experienced reviewers use a broader variety of explanations suggesting that experience and skills likely play a role for effective explanation.

5.1.2 LLMs Can Help Reviewers Generate Alternative Explanations. Providing suitable explanations requires effort from reviewers. To reduce this, reviewers could use LLMs to generate alternative explanations. By using the seven explanation categories identified in our work, reviewers can reason and structure their feedback more efficiently, making it easier for authors to understand their suggestions and intentions. For instance, if a code author continues to have questions after the initial explanation, a reviewer might use an LLM to generate an alternative explanation. The various explanation types serve as different "templates" for the LLM to create review comments depending on the reviewers' needs. Additionally, in RQ3 we found that experienced reviewers tend to give more diversified explanations compared to novices. This suggests that LLMs could also help novices in creating other types of explanations. Our prompts and the tool discussed in Section 4 can serve as a first step to this end.

5.1.3 Authors Can Proactively Seek Explanations and Eventually Be Assisted by LLMs Too. Through a proactive approach, authors can gain a better and quicker understanding of not only what aspects require improvement but also the rationale behind these suggested enhancements in a code review. This empowers authors to make well-informed decisions and, e.g., implement modifications that align with the project goals and standards. Code authors might specify the kind of explanation they miss, such as a similar example they can look at. This can lead to need-driven knowledge sharing and to improvement of the communication effectiveness among developers.

In particular, authors can themselves use LLMs to tailor the type of explanations needed. Authors can utilize this method to get tailored explanations, potentially aiding a more nuanced understanding of the feedback. Our evaluation results of ChatGPT in transforming explanations show the potential for integrating such an LLM-driven approach into the code review workflow. Instead of waiting for or getting a single explanation, authors could generate and explore multiple explanations. This can minimize back-and-forth communication between authors and reviewers, which in turn would speed up the review process, particularly in continuous deployment settings. In the broader context, this can contribute to better on-boarding of new contributors [62] and to reducing pull requests abandonment [35]. In fact, Khatoonabadi et al. [35] conducted a mixed-method study of 4,450 abandoned pull requests and found that lengthy reviews are a contributing factor leading authors to stop working on a change. When analyzing the reasons behind abandonment, the researchers found that the leading cause is that the authors had difficulty addressing maintainers' comments.

5.2 Implications for Future Research

5.2.1 Understanding Explanation Needs and Styles as Part of Knowledge Sharing. Our work only provides starting evidence about the prevalence and typology of explanations in code reviews. Future researchers should investigate how various explanation types and "explanation styles" in code reviews affect the review outcome and the overall review process. For instance, it remains unclear how specific types of explanations, their timing, or order impact code quality, developer engagement, and the efficiency of collaboration. Answering these questions likely provides insights for improving code review practices. Moreover, it is also worth investigating what influences the explanation needs [6] (and preferences) of both the reviewers and authors. We took a first step in this direction analyzing the difference between novices and experienced reviewers. Gathering additional data about the reviewers or using different research methods (as controlled experiment or interviews) can provide a more thorough understanding toward a personalized, need-driven code review explanations and knowledge sharing between developers in general [45].

Generally, explanations (i.e., rationale or answers to *why* questions) represent a valuable type of development knowledge that often remains in the head of developers (tacit knowledge) [46, 47]. Our work can help identify—and eventually externalize—this knowledge from code reviews for reusing it in other development tasks [27, 45]. For instance, explanations mentioning coding rules and practices to follow or test cases to cover could potentially be useful for new project members [62] and other developers to extract and share in the documentation. Since manually writing informative review comments is time-intensive, future researchers should focus on developing automated solutions to generate specific explanation types directly from the code context.

5.2.2 What Makes a Good Explanation? Our work stops at the reviewer's perspective and does not investigate whether and when explanations are actually effective—that is whether a certain explanation is of a high or low quality. While researchers have investigated the quality of software documentation in the past [46, 66], explanations in review comments are rather informal and different. Therefore, it remains unclear what makes a good explanation.

One potential influencing factor is the preciseness and quality of the language used. Another is the combination of explanations and arguments—an aspect this work does not address. Our categorization was based on the entire review comment. When analyzing explanations, we observed that in some cases several explanation categories are combined in one comment. Assuming that multi-typed explanations would likely reside in lengthier comments, we also sampled comments that have a text length above the 75th percentile. Out of the 71 long code review comments, we found only nine that have multiple types of explanations. For example: "We have another memory leak here. This code should be replaced with: 'int sizes = features.size(); TfLiteInterpreterResizeInputTensor(interpreter, 0, & sizes, 1); TfLiteInterpreterAllocateTensors(interpreter);' That way we don't need to free the memory as it will be allocated on the stack and freed when the function exits."[17] In this comment, the reviewer first points to a problem—a memory leak (Category 6). The reviewer then proposes a specific code change, followed by an explanation of the benefits (Category 7). However, when considering the entire code review comment, the primary focus is on identifying the issue (i.e., the memory leak). The subsequent sentences, including the suggestions and their benefits, serve to reinforce the resolution of the initially stated issue. Thus, in RQ1, this entire comment is categorized under Category 6. We included these nine cases with multiple types of explanations in our replication package. This observation opens up the possibility of combining various types of explanations to provide more comprehensive–possibly more useful—feedback. Future studies can investigate when an explanation should be short or more detailed and when it should be single-typed or multi-typed. Strategies for how to effectively combine explanations into single review comments are yet to be investigated.

5.3 Threats to Validity

5.3.1 Internal Validity. A potential threat to internal validity is our selection approach for useful code reviews. For this, we followed the previous work by Bosu et al. [5]. We also adopted the method proposed by Zhang et al. [82] as an alternative to the original internal sentiment analysis tool, which was not accessible at the time of this research. While this adaptation may introduce additional noise, we believe this to be minimal. Zhang et al. [82] demonstrated through extensive evaluation that their method outperforms five existing approaches across six datasets. We make our implementation of the decision tree as well as our script to retrieve Gerrit code reviews public [76], to enable other researchers to validate our findings.

[17] https://android-review.googlesource.com/c/platform/frameworks/opt/gamesdk/+/2238874/10..11/src/memory_advice/core/predictor.cpp#b92

Another internal validity concern is a potential mislabeling in our data. To ensure the quality of the labeling, we manually analyzed the data over multiple iterations. We revisited categorizations several times over a year—in December 2022, April 2023, August 2023, and June 2024. However, it remains possible that we mislabeled some examples. We also did not report the inter-rater agreement for the explanation categories. We instead employed a collaborative approach (open card sorting) that allowed us to discuss and resolve any differences immediately during the categorization process. As discrepancies were inherently resolved through discussion until a unanimous decision was reached, we cannot calculate the inter-rater agreement score. We included our labeled code review dataset as part of the replication package so other researchers can validate our findings.

5.3.2 External Validity. The generalizability of our findings is subject to potential threats too. One potential risk arises from the scope of our evaluations on the ability of ChatGPT to generate specific types of explanations. In the first phase of RQ4 analysis in our study, we assessed 120 samples for accuracy and correctness. This was followed by an evaluation of 90 samples in the second phase (generated using the best-performing prompt from phase one) to assess correctness, type correctness, clarity, and usefulness. The combined efforts for both phases total approximately 115 hours. Should we attempt to extend these evaluations to encompass all generated samples, the first phase alone would require an estimated 1,192 hours per evaluator. This figure is derived from the assessment of 298 original categories, each expanded through transformations into five additional categories and evaluated across four different prompts, resulting in a total of 7,152 samples, with each requiring about 10 minutes for assessment. Due to these constraints, previous studies also have often utilized relatively small sample sizes for similar evaluations [7, 12, 34]. For instance, Geng et al. [12] evaluated 100 samples in their study.

Furthermore, we focused our analysis on all Google projects available on Gerrit. We selected Google projects because Google's code review guidelines specifically highlight the necessity of explanation in code review comments. Although we studied multiple projects, the results may not be generalizable to projects using other code review platforms. Furthermore, to maintain high quality and relevance in our manual analysis, we filtered out only the useful code review comments. However, it is important to note that while some reviews may not have been deemed useful through automatic filtering, they may still contain valuable insights and explanations.

5.3.3 Construct Validity. A potential threat to construct validity arises from the variability in ChatGPT-generated explanations caused by different prompts. To create our prompts, we analyzed patterns from different types of explanations and followed the approaches from previous studies [79, 84] that used prompts for ChatGPT. While different prompts and models may produce different explanations, we think that the overall trends and findings of our work remain valid.

6 Conclusion and Future Direction

Code review is a central activity in software development. While numerous studies have attempted to automate code review activities, there has not been much work on understanding how reviewers actually bring their points across by explaining their feedback to authors. This study aims to fill this gap by taking a close look at reviewer feedback in code review comments.

We collected a useful code review dataset from Gerrit and analyzed how often reviewers used explanations in their first review comments. We found that many of the comments in our data (i.e., 42%) only contain improvement suggestions without explanations. At the same time, the majority (79%) of review explanations are accompanied by a suggestion. When manually analyzing these explanations we identified seven distinct categories. We found that reviewers typically use Category 6, where they state an issue with the code under review. When correlating developers' experience with the explanations, we found that experienced reviewers are more likely to use diverse

categories and to discuss potential future implications when explaining their review suggestions. We also evaluated how well can ChatGPT transform different types of explanations using different prompts. Our results show that in 88 and 89 out of 90 cases, ChatGPT successfully generated a correct explanation and an explanation of the intended type, respectively. Moreover, the generated explanations demonstrated a high level of informativeness and clarity.

For future work, we plan to leverage LLMs not only to suggest different types of explanations but also the full code review comment (which includes the identification of the target location and type of explanation needed). We also plan to explore scenarios where an explanation is not initially provided during a code review, and the developer subsequently requests clarification. Specifically, we intend to investigate the effectiveness of prompting ChatGPT directly with the developer's question to generate the required explanation. Finally, substantial empirical work is needed to gather additional insights from developers (e.g., through interviews and experiments) regarding their explanation needs, styles, and investigate which explanations are most useful in which context.

References

[1] Meta AI. 2024. Meta LLaMA. Retrieved from https://ai.meta.com/blog/meta-llama-3/

[2] Mistral AI. 2024. Mixtral of experts. Retrieved from https://mistral.ai/news/mixtral-of-experts/

[3] Alberto Bacchelli and Christian Bird. 2013. Expectations, outcomes, and challenges of modern code review. In *Proceedings of the 2013 35th International Conference on Software Engineering (ICSE '13)*. IEEE, 712–721.

[4] Gabriele Bavota and Barbara Russo. 2015. Four eyes are better than two: On the impact of code reviews on software quality. In *Proceedings of the 2015 IEEE International Conference on Software Maintenance and Evolution (ICSME '15)*. IEEE, 81–90.

[5] Amiangshu Bosu, Michaela Greiler, and Christian Bird. 2015. Characteristics of useful code reviews: An empirical study at Microsoft. In *Proceedings of the 2015 IEEE/ACM 12th Working Conference on Mining Software Repositories*. IEEE, 146–156.

[6] Abir Bouraffa and Walid Maalej. 2020. Two decades of empirical research on developers' information needs: A preliminary analysis. In *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops*, 71–77.

[7] Songqiang Chen, Xiaoyuan Xie, Bangguo Yin, Yuanxiang Ji, Lin Chen, and Baowen Xu. 2020. Stay professional and efficient: automatically generate titles for your bug reports. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, 385–397.

[8] Codeflow. 2016. Codeflow—Modern, light-weight API integration platform. Retrieved February 8, 2023 from https://codeflow.co/

[9] Paul A. David and Joseph S. Shapiro. 2008. Community-based production of open-source software: What do we know about the developers who participate? *Information Economics and Policy* 20, 4 (2008), 364–398. DOI: https://doi.org/10.1016/j.infoecopol.2008.10.001

[10] Felipe Ebert, Fernando Castor, Nicole Novielli, and Alexander Serebrenik. 2017. Confusion detection in code reviews. In *Proceedings of the 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME '17)*. IEEE, 549–553.

[11] William T. Gattrell, Patricia Logullo, Esther J. van Zuuren, Amy Price, Ellen L. Hughes, Paul Blazey, Christopher C. Winchester, David Tovey, Keith Goldman, Amrit Pali Hungin, et al. 2024. ACCORD (ACcurate COnsensus reporting document): A reporting guideline for consensus methods in biomedicine developed via a modified Delphi. *PLoS Medicine* 21, 1 (2024), e1004326.

[12] M. Geng, S. Wang, D. Dong, H. Wang, G. Li, Z. Jin, X. Mao, and X. Liao. 2024. Large language models are Few-shot summarizers: Multi-intent comment generation via in-context learning. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 1–13.

[13] Github. 2023. About pull request reviews - GitHub docs. Retrieved February 8, 2023 from https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/reviewing-changes-in-pull-requests/about-pull-request-reviews

[14] Google. 2019. How to write code review comments —Eng-practices. Retrieved February 8, 2023 from https://google.github.io/eng-practices/review/reviewer/comments.html

[15] Google. 2023. Android gerrit. Retrieved February 8, 2023 from https://android-review.googlesource.com

[16] Google. 2023. Bazel gerrit. Retrieved February 8, 2023 from https://bazel-review.googlesource.com

[17] Google. 2023. Chromium gerrit. Retrieved February 8, 2023 from https://chromium-review.googlesource.com

[18] Google. 2023. Dart gerrit. Retrieved February 8, 2023 from https://dart-review.googlesource.com
[19] Google. 2023. Flutter gerrit. Retrieved February 8, 2023 from https://flutter-review.googlesource.com
[20] Google. 2023. Fuschia gerrit. Retrieved February 8, 2023 from https://fuschia-review.googlesource.com
[21] Google. 2023. Gerrit. Retrieved February 8, 2023 from https://gerrit-review.googlesource.com
[22] Google. 2023. Gerrit: Google open source projects. Retrieved February 8, 2023 from https://opensource.google/projects/gerrit
[23] Google. 2023. Go gerrit. Retrieved February 8, 2023 from https://go-review.googlesource.com
[24] Sanuri Gunawardena, Ewan Tempero, and Kelly Blincoe. 2023. Concerns identified in code review: A fine-grained, faceted classification. *Information and Software Technology* 153 (2023), 107054.
[25] Qi Guo, Junming Cao, Xiaofei Xie, Shangqing Liu, Xiaohong Li, Bihuan Chen, and Xin Peng. 2024. Exploring the potential of ChatGPT in automated code refinement: An empirical study. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 1–13.
[26] Anshul Gupta and Neel Sundaresan. 2018. Intelligent code reviews using deep learning. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '18) Deep Learning Day*.
[27] Hans-Jörg Happel and Walid Maalej. 2008. Potentials and challenges of recommendation systems for software development. In *Proceedings of the 2008 International Workshop on Recommendation Systems for Software Engineering*, 11–15.
[28] Masum Hasan, Anindya Iqbal, Mohammad Rafid Ul Islam, A. J. M. Imtiajur Rahman, and Amiangshu Bosu. 2021. Using a balanced scorecard to identify opportunities to improve code review effectiveness: An industrial experience report. *Empirical Software Engineering* 26 (2021), 1–34.
[29] Yang Hong, Chakkrit Tantithamthavorn, Patanamon Thongtanunam, and Aldeida Aleti. 2022. Commentfinder: A simpler, faster, more accurate code review comments recommendation. In *Proceedings of the 30th ACM joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 507–519.
[30] Fan Huang, Haewoon Kwak, and Jisun An. 2023. Is ChatGPT better than human annotators? Potential and limitations of ChatGPT in explaining implicit hate speech. In *Companion Proceedings of the ACM Web Conference 2023 (WWW '23 Companion)*. ACM, New York, NY, 294–297. DOI: https://doi.org/10.1145/3543873.3587368
[31] Faria Huq, Masum Hasan, Md Mahim Anjum Haque, Sazan Mahbub, Anindya Iqbal, and Toufique Ahmed. 2022. Review4Repair: Code review aided automatic program repairing. *Information and Software Technology* 143 (2022), 106765.
[32] Shengbei Jiang, Jiabao Zhang, Wei Chen, Bo Wang, Jianyi Zhou, and Jie M. Zhang. 2024. Evaluating fault localization and program repair capabilities of existing closed-source general-purpose LLMs. In *Proceedings of the 1st International Workshop on Large Language Models for Code*, ACM, New York, NY, 75–78. DOI: https://doi.org/10.1145/3643795.3648390
[33] Arthur Kamienski and Cor-Paul Bezemer. 2021. An empirical study of Q & A websites for game developers. *Empirical Software Engineering* 26, 6 (2021), 115. DOI: https://doi.org/10.1007/s10664-021-10014-4
[34] Hong Jin Kang, Fabrice Harel-Canada, Muhammad Ali Gulzar, Violet Peng, and Miryung Kim. 2024. Human-in-the-loop synthetic text data inspection with provenance tracking. In *Findings of the Association for Computational Linguistics: NAACL 2024*. Duh Kevin, Gomez Helena, and Bethard Steven (Eds.), Association for Computational Linguistics, Mexico, 3118–3129. Retrieved from https://aclanthology.org/2024.findings-naacl.197
[35] SayedHassan Khatoonabadi, Diego Elias Costa, Rabe Abdalkareem, and Emad Shihab. 2023. On wasted contributions: Understanding the dynamics of contributor-abandoned pull requests–A mixed-methods study of 10 large open-source projects. *ACM Transactions on Software Engineering and Methodology* 32, 1 (2023), 1–39.
[36] Oleksii Kononenko, Olga Baysal, and Michael W. Godfrey. 2016. Code review quality: How developers see it. In *Proceedings of the 38th International Conference on Software Engineering*, 1028–1038.
[37] Oleksii Kononenko, Olga Baysal, Latifa Guerrouj, Yaxin Cao, and Michael W. Godfrey. 2015. Investigating code review quality: Do people and participation matter? In *Proceedings of the 2015 IEEE International Conference on Software Maintenance and Evolution (ICSME '15)*. IEEE, 111–120.
[38] Makrina Viola Kosti, Robert Feldt, and Lefteris Angelis. 2014. Personality, emotional intelligence and work preferences in software engineering: An empirical study. *Information and Software Technology* 56, 8 (2014), 973–990. DOI: https://doi.org/10.1016/j.infsof.2014.03.004
[39] Lingwei Li, Li Yang, Huaxi Jiang, Jun Yan, Tiejian Luo, Zihan Hua, Geng Liang, and Chun Zuo. 2022. AUGER: Automatically generating review comments with pre-training models. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 1009–1021.
[40] Zhiyu Li, Shuai Lu, Daya Guo, Nan Duan, Shailesh Jannu, Grant Jenks, Deep Majumder, Jared Green, Alexey Svyatkovskiy, Shengyu Fu, et al. 2022. Automating code review activities by large-scale pre-training. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 1035–1047.

[41] Jingjing Liang, Yaozong Hou, Shurui Zhou, Junjie Chen, Yingfei Xiong, and Gang Huang. 2019. How to explain a patch: An empirical study of patch explanations in open source projects. In *Proceedings of the 2019 IEEE 30th International Symposium on Software Reliability Engineering (ISSRE '19)*. IEEE, 58–69.

[42] Brian Y. Lim, Qian Yang, Ashraf M. Abdul, and Danding Wang. 2019. Why these explanations? Selecting intelligibility types for explanation goals. In *Joint Proceedings of the ACM IUI 2019 Workshops*, Vol. 2327.

[43] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2024. Is your code generated by chatgpt really correct? Rigorous evaluation of large language models for code generation. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, 21558–21572.

[44] Yue Liu, Thanh Le-Cong, Ratnadira Widyasari, Chakkrit Tantithamthavorn, Li Li, Xuan-Bach D. Le, and David Lo. 2024. Refining ChatGPT-generated code: Characterizing and mitigating code quality issues. *ACM Transactions on Software Engineering and Methodology* 33, 5, Article 116 (2024), 1–26.

[45] Walid Maalej. 2023. From RSSE to BotSE: Potentials and challenges revisited after 15 years. In *Proceedings of the 2023 IEEE/ACM 5th International Workshop on Bots in Software Engineering (BotSE '23)*, 19–22. DOI: https://doi.org/10.1109/BotSE59190.2023.00012

[46] Walid Maalej and Martin P. Robillard. 2013. Patterns of knowledge in API reference documentation. *IEEE Transactions on Software Engineering* 39, 9 (2013), 1264–1282.

[47] Walid Maalej, Rebecca Tiarks, Tobias Roehm, and Rainer Koschke. 2014. On the comprehension of program comprehension. *ACM Transactions on Software Engineering and Methodology* 23, 4 (2014), 1–37.

[48] Stephen MacNeil, Andrew Tran, Arto Hellas, Joanne Kim, Sami Sarsa, Paul Denny, Seth Bernstein, and Juho Leinonen. 2023. Experiences from using code explanations generated by large language models in a web software development E-book. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*, 931–937.

[49] Shane McIntosh, Yasutaka Kamei, Bram Adams, and Ahmed E. Hassan. 2014. The impact of code review coverage and code review participation on software quality: A case study of the Qt, VTK, and itk projects. In *Proceedings of the 11th Working Conference on Mining Software Repositories*, 192–201.

[50] Rodrigo Morales, Shane McIntosh, and Foutse Khomh. 2015. Do code review practices impact design quality? A case study of the Qt, Vtk, and itk projects. In *Proceedings of the 2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER '15)*. IEEE, 171–180.

[51] OpenAI. 2023. ChatGPT version. Retrieved February 8, 2023 from https://help.openai.com/en/articles/6825453-chatgpt-release-notes

[52] Matheus Paixao, Jens Krinke, Donggyun Han, and Mark Harman. 2018. CROP: Linking code reviews to source code changes. In *Proceedings of the 15th International Conference on Mining Software Repositories*, 46–49.

[53] Luca Pascarella, Davide Spadini, Fabio Palomba, Magiel Bruntink, and Alberto Bacchelli. 2018. Information needs in contemporary code review. *Proceedings of the ACM on Human-Computer Interaction* 2, CSCW (2018), 1–27.

[54] Phacility. 2021. Phacility - Phabricator. Retrieved February 8, 2023 from https://phacility.com/phabricator/

[55] Chris Quirk, Pallavi Choudhury, Jianfeng Gao, Hisami Suzuki, Kristina Toutanova, Michael Gamon, Scott Wen-tau Yih, Lucy Vanderwende, and Colin Cherry. 2012. MSR SPLAT, a language analysis toolkit. In *Proceedings of the 2012 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Demonstration Session*, 21–24.

[56] Mohammad Masudur Rahman, Chanchal K. Roy, and Raula G. Kula. 2017. Predicting usefulness of code review comments using textual features and developer experience. In *Proceedings of the 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR '17)*. IEEE, 215–226.

[57] Shadikur Rahman, Umme Ayman Koana, and Maleknaz Nayebi. 2022. Example driven code review explanation. In *Proceedings of the 16th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, 307–312.

[58] Google Research. 2024. Google Gemini. Retrieved from https://gemini.google.com

[59] Leocadio Rodríguez-Mañas, Catherine Féart, Giovanni Mann, Jose Viña, Somnath Chatterji, Wojtek Chodzko-Zajko, Magali Gonzalez-Colaço Harmand, Howard Bergman, Laure Carcaillon, Caroline Nicholson, et al. 2013. Searching for an operational definition of frailty: A Delphi method based consensus statement. The frailty operative definition-consensus conference project. *Journals of Gerontology Series A: Biomedical Sciences and Medical Sciences* 68, 1 (2013), 62–67.

[60] Caitlin Sadowski, Emma Söderberg, Luke Church, Michal Sipko, and Alberto Bacchelli. 2018. Modern code review: A case study at Google. In *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice*, 181–190.

[61] Donna Spencer. 2009. *Card Sorting: Designing Usable Categories*. Rosenfeld Media.

[62] Christoph Stanik, Lloyd Montgomery, Daniel Martens, Davide Fucci, and Walid Maalej. 2018. A simple NLP-based approach to support onboarding and retention in open source communities. In *Proceedings of the 2018 IEEE International Conference on Software Maintenance and Evolution (ICSME '18)*. IEEE, 172–182.

[63] Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Haijun Wang, Zhengzi Xu, Xiaofei Xie, and Yang Liu. 2024. Gptscan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 1–13.

[64] Davide Taibi, Andrea Janes, and Valentina Lenarduzzi. 2017. How developers perceive smells in source code: A replicated study. *Information and Software Technology* 92 (2017), 223–235. DOI: https://doi.org/10.1016/j.infsof.2017.08.007

[65] Ronald J. Tallarida and Rodney B. Murray. 1987. Chi-square test. In *Manual of Pharmacologic Calculations: With Computer Programs*. Springer, New York, NY, 140–142. DOI: https://doi.org/10.1007/978-1-4612-4974-0_43

[66] Henry Tang and Sarah Nadi. 2023. Evaluating software documentation quality. In *Proceedings of the 2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR '23)*, 67–78. DOI: https://doi.org/10.1109/MSR59073.2023.00023

[67] Patanamon Thongtanunam, Chanathip Pornprasit, and Chakkrit Tantithamthavorn. 2022. AutoTransform: Automated code transformation to support modern code review process. In *Proceedings of the 44th International Conference on Software Engineering*, 237–248.

[68] Patanamon Thongtanunam, Chakkrit Tantithamthavorn, Raula Gaikovina Kula, Norihiro Yoshida, Hajimu Iida, and Ken-ichi Matsumoto. 2015. Who should review my code? A file location-based code-reviewer recommendation approach for modern code review. In *Proceedings of the 2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER '15)*. IEEE, 141–150.

[69] Michele Tufano, Jevgenija Pantiuchina, Cody Watson, Gabriele Bavota, and Denys Poshyvanyk. 2019. On learning meaningful code changes via neural machine translation. In *Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE '19)*. IEEE, 25–36.

[70] Rosalia Tufano, Simone Masiero, Antonio Mastropaolo, Luca Pascarella, Denys Poshyvanyk, and Gabriele Bavota. 2022. Using pre-trained models to boost code review automation. In *Proceedings of the 44th International Conference on Software Engineering*, 2291–2302.

[71] Rosalia Tufano, Luca Pascarella, Michele Tufano, Denys Poshyvanyk, and Gabriele Bavota. 2021. Towards automating code review activities. In *Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE '21)*. IEEE, 163–174.

[72] Asif Kamal Turzo and Amiangshu Bosu. 2023. What makes a code review useful to OpenDev developers? An empirical investigation. arXiv:2302.11686. Retrieved from https://arxiv.org/abs/2302.11686

[73] Anderson Uchôa, Caio Barbosa, Daniel Coutinho, Willian Oizumi, Wesley K. G. Assunçao, Silvia Regina Vergilio, Juliana Alves Pereira, Anderson Oliveira, and Alessandro Garcia. 2021. Predicting design impactful changes in modern code review: A large-scale empirical study. In *Proceedings of the 2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR '21)*. IEEE, 471–482.

[74] Danding Wang, Qian Yang, Ashraf Abdul, and Brian Y. Lim. 2019. Designing theory-driven user-centric explainable AI. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, 1–15.

[75] Ratnadira Widyasari, Jia Wei Ang, Truong Giang Nguyen, Neil Sharma, and David Lo. 2024. Demystifying faulty code with LLM: Step-by-step reasoning for explainable fault localization. In *Proceedings of the 2024 IEEE International Conference on Software Analysis, Evolution, and Reengineering (SANER '24)*. IEEE, 568–579.

[76] Ratnadira Widyasari, Ting Zhang, Abir Bouraffa, Walid Maalej, and David Lo. 2023. Code review explanation replication package. Retrieved February 8, 2023 from https://figshare.com/s/135201b8f87ab705448b

[77] Pavlína Wurzel Gonçalves, Gül Çalikli, and Alberto Bacchelli. 2022. Interpersonal conflicts during code review: Developers' experience and practices. *Proceedings of the ACM on Human-Computer Interaction* 6, CSCW1 (2022), 1–33.

[78] Pavlína Wurzel Gonçalves, Gül Calikli, Alexander Serebrenik, and Alberto Bacchelli. 2023. Competencies for code review. *Proceedings of the ACM on Human-Computer Interaction* 7, CSCW1 (2023), 1–33.

[79] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using chatgpt. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '24)*. ACM, New York, NY, 819–831. DOI: https://doi.org/10.1145/3650212.3680323

[80] Bowen Xu, Le An, Ferdian Thung, Foutse Khomh, and David Lo. 2020. Why reinventing the wheels? An empirical study on library reuse and re-implementation. *Empirical Software Engineering* 25 (2020), 755–789.

[81] Motahareh Bahrami Zanjani, Huzefa Kagdi, and Christian Bird. 2015. Automatically recommending peer reviewers in modern code review. *IEEE Transactions on Software Engineering* 42, 6 (2015), 530–543.

[82] Ting Zhang, Bowen Xu, Ferdian Thung, Stefanus Agus Haryono, David Lo, and Lingxiao Jiang. 2020. Sentiment analysis for software engineering: How far can pre-trained transformer models go? In *Proceedings of the 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME '20)*. IEEE, 70–80. Retrieved February 8, 2023 from https://github.com/soarsmu/SA4SE/

[83] Xin Zhou, Ting Zhang, and David Lo. 2024. Large language model for vulnerability detection: Emerging results and future directions. In *Proceedings of the 2024 International Conference on Software Engineering (ICSE '24), New Ideas and Emerging Results (NIER'24) Track*. IEEE, 47–51.
[84] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2023. Large language models are Human-level prompt engineers. In *The Eleventh International Conference on Learning Representations*. Retrieved from https://openreview.net/forum?id=92gvk82DE-

Received 3 November 2023; revised 10 October 2024; accepted 3 December 2024